Posgrado Gemini: Arquitectura, Desarrollo e Integración

https://gemini.google.com/app/454a59f76dd87fc9

User prompt: Que enseñar a los estudiantes de posgrado ia de Gemini

Response: Un programa formativo de IA para posgrado enfocado en el ecosistema de Google Gemini debe ir más allá del uso superficial de *prompts* y abordar la arquitectura, el desarrollo técnico y la implementación estratégica.

Fundamentos Técnicos y Arquitectura Multi-modal

- **Arquitectura Transformer Avanzada:** Modelado de secuencias, mecanismos de atención nativa para texto, audio, visión y código simultáneamente.
- **Procesamiento de Contexto Extremo:** Gestión y optimización de ventanas de contexto ultra-largas (1M+ tokens), técnicas de reducción de costos y *needle in a haystack testing*.
- **Evaluación y Benchmarking:** Uso de métricas académicas y métricas de producción para evaluar sesgos, alucinaciones y precisión factual.

Ingeniería de Prompts y Razonamiento Avanzado

- **Estrategias de Razonamiento:** Implementación de *Chain-of-Thought* (CoT), *Tree-of-Thoughts* (ToT) y activación de modelos con razonamiento profundo (*Deep Think*).
- **Diseño Sistemático de Prompts:** Estructuración de instrucciones para análisis cualitativo, síntesis bibliográfica y generación de código crítico.
- **Control de Salida:** Forzado de formatos estructurados (JSON schemas, llamadas a funciones/tools) para integración con pipelines de software.

Desarrollo e Integración con la API de Gemini

- **Google GenAI SDK:** Desarrollo práctico en Python y TypeScript utilizando la API oficial de Gemini.
- **Retrieval-Augmented Generation (RAG):** Construcción de arquitecturas RAG avanzadas utilizando embeddings de Google, bases de datos vectoriales y búsqueda híbrida.
- **Function Calling y Agentic Workflows:** Integración de herramientas externas (APIs, calculadoras, ejecución de código en sandbox) para crear agentes autónomos.

Ecosistema y Herramientas Especializadas

Herramienta / Tecnología	Enfoque Académico y Práctico
NotebookLM / Gemini Notebook	Gestión de fuentes de investigación, síntesis de literatura científica y generación de Resúmenes de Audio/Video.
Google AI Studio	Prototipado rápido de prompts, ajuste de hiperparámetros (temperatura, Top-K, Top-P) y exportación de código.
Vertex AI	Despliegue empresarial, Fine-Tuning de modelos Gemini, monitoreo de drift y gobernanza de datos.
Google Workspace & Extensions	Automatización de flujos de trabajo científicos mediante integración con Docs, Sheets y Drive.

Ética, Gobernanza y Responsabilidad

- **Principios de IA Responsable de Google:** Evaluación de riesgos de ciberseguridad, equidad y mitigación de sesgos algorítmicos.
- **Privacidad de Datos e Propiedad Intelectual:** Manejo de datos confidenciales en entornos académicos y corporativos; marcas de agua digitales (SynthID) y procedencia de contenido.
- **Integridad Académica:** Normativa sobre atribución, transparencia en la coautoría de investigaciones y detección de contenido generado.

User prompt: Diseña un syllabus detallado de 16 semanas para esta asignatura de posgrado sobre Gemini e IA Generativa.

Response: **Especialización en IA Generativa: Arquitectura, Desarrollo y Aplicaciones con Google Gemini**

- **Nivel:** Posgrado (Maestría / Doctorado)
- **Duración:** 16 semanas (48 horas lectivas + trabajo práctico)
- **Requisitos previos:** Programación intermedia en Python, fundamentos de Machine Learning y gestión de APIs REST.

Módulo 1: Fundamentos y Arquitectura Nativa Multimodal (Semanas 1-4)

- **Semana 1: Introducción a la Era Multimodal y Ecosistema Gemini**
  - Evolución de los LLMs a LMMs (Large Multimodal Models).
  - Filosofía de diseño: Entrenamiento nativo multimodal vs. adaptación tardía (*early fusion vs. late fusion*).
  - Visión general de modelos: Gemini Ultra, Pro, Flash y Nano.
  - *Práctica:* Configuración del entorno con Google GenAI SDK en Python y primeros llamadas a API.
- **Semana 2: Mecanismos de Atención y Arquitectura Interna**
  - Mecanismos de atención en secuencias mixtas (texto, audio, imagen, video).
  - Tokenización cruzada y representación latente unificada.
  - Evaluación de modelos de lenguaje: Métricas estándar (BLEU, ROUGE) vs. *LLM-as-a-Judge*.
  - *Práctica:* Benchmark cualitativo y cuantitativo comparando distintas versiones de Gemini sobre datasets complejos.
- **Semana 3: Ventanas de Contexto Masivo (Long-Context Engineering)**
  - Arquitectura detrás de la ventana de contexto de 1M+ a 2M de tokens.
  - Técnicas de evaluación: *Needle in a Haystack* (NIAH) y pérdida de atención intermedia (*lost in the middle*).
  - Implicaciones en costos, latencia y estrategias de caché de contexto (*Context Caching*).
  - *Práctica:* Implementación de análisis completo de repositorios de código o libros enteros mediante Context Caching.
- **Semana 4: Prototipado Rápido con Google AI Studio**
  - Exploración de hiperparámetros: Temperatura, Top-K, Top-P, Penalización de frecuencia y presencia.
  - Formateo de salida estricto: JSON Schema y respuestas estructuradas nativas.
  - Exportación de prototipos a código listo para producción.
  - *Práctica:* Diseño de un pipeline en Google AI Studio para extracción de datos no estructurados a JSON estricto.

Módulo 2: Ingeniería de Prompts Avanzada y Razonamiento (Semanas 5-7)

- **Semana 5: Patrones Avanzados de Prompting**
  - Systematic Prompt Engineering: System Instructions, Few-Shot In-Context Learning, y Meta-Prompting.
  - Delimitación de contextos y mitigación de inyecciones de prompt (*Prompt Injection*).
  - *Práctica:* Construcción de un framework de evaluación de robustez de prompts.
- **Semana 6: Arquitecturas de Razonamiento Complejo**
  - *Chain-of-Thought* (CoT), *Self-Consistency*, *Tree-of-Thoughts* (ToT) y *Graph-of-Thoughts*.
  - Modelos de razonamiento profundo y planificación iterativa.
  - *Práctica:* Implementación paso a paso de un solucionador de problemas matemáticos y lógicos utilizando ToT sobre la API de Gemini.
- **Semana 7: Procesamiento Nativo de Imagen, Audio y Video**
  - Prompting multimodal: Extracción de información de video cuadros por cuadro (*sampling rate*).
  - Análisis de audio sin transcripción previa e inspección visual de diagramas y planos.
  - *Práctica:* Creación de un asistente que analiza una clase grabada en video y genera un resumen interactivo con marcas de tiempo.

Módulo 3: Sistemas RAG y Agentes Autónomos (Semanas 8-11)

- **Semana 8: Embeddings y Búsqueda Vectorial**
  - Modelos de Embeddings de Google (texto e imagen).
  - Bases de datos vectoriales (Chroma, Pinecone, Vertex AI Vector Search).
  - Estrategias de chunking multimodal y distancia semántica.
  - *Práctica:* Generación de un espacio latente para documentos científicos en PDF que contienen texto y figuras.
- **Semana 9: RAG Avanzado (Retrieval-Augmented Generation)**

- Arquitectura RAG: Re-ranking, Hybrid Search (léxico + semántico) y Query Transformation.
- Integración con Google Search Grounding para respuestas con datos en tiempo real.
- *Práctica:* Desarrollo de un sistema RAG sobre una base de datos médica con citación de fuentes y verificación de hechos.

- **Semana 10: Calling de Funciones (Function Calling) y Herramientas**
  - Declaración de schemas OpenAPI/JSON para herramental externo.
  - Ejecución de código en sandbox y uso de calculadoras u ORMs mediante el modelo.
  - *Práctica:* Conexión de Gemini con una API meteorológica y una base de datos SQL mediante Function Calling.
- **Semana 11: Arquitecturas de Agentes y ReAct**
  - El patrón ReAct (Reason + Act) e integración con LangChain / LlamaIndex / AutoGen.
  - Agentes multimodales y sistemas multi-agente con roles especializados.
  - *Práctica:* Implementación de un agente investigador autónomo que busque información, ejecute código en Python para validar datos y redacte un informe final.

Módulo 4: Despliegue Empresarial y Producción (Semanas 12-14)

- **Semana 12: MLOps con Enterprise Gemini en Vertex AI**
  - Diferencias entre Google AI Studio y Google Cloud Vertex AI.
  - Despliegue seguro, IAM, cuotas y orquestación de llamadas.
  - Monitoreo de latencia, costos y deriva del modelo (*drift*).
  - *Práctica:* Despliegue de un endpoint de producción en Vertex AI con autenticación por Service Accounts.
- **Semana 13: Fine-Tuning y Model Alignment**
  - Parameter-Efficient Fine-Tuning (PEFT), LoRA y QLoRA aplicados a modelos de la familia Gemini.
  - RLHF (Reinforcement Learning from Human Feedback) y DPO (Direct Preference Optimization).
  - *Práctica:* Ajuste fino de un modelo Gemini Nano o Flash para adaptar el tono y estilo a un dominio legal/médico específico.
- **Semana 14: Extensibilidad y Herramientas de Investigación**
  - Flujos de trabajo científicos con NotebookLM y Gemini Extensions.
  - Automatización de tareas de investigación académica utilizando Google Workspace APIs.
  - *Práctica:* Desarrollo de un flujo de revisión de literatura científica automatizado.

Módulo 5: Ética, Seguridad y Proyecto Final (Semanas 15-16)

- **Semana 15: IA Responsable, Seguridad y Marcas de Agua**
  - Filtros de seguridad nativos (*Safety Settings*) y ajuste de umbrales.
  - Detección de contenido sintético y marcas de agua invisibles (SynthID).
  - Sesgos en IA multimodal, privacidad y leyes de gobernanza (EU AI Act, GDPR).
  - *Práctica:* Auditoría de seguridad y sesgos sobre un sistema conversacional desplegado.
- **Semana 16: Presentación de Proyectos de Investigación Finales**
  - Presentación tipo conferencia (paper corto + demostración de software en vivo).
  - Evaluación por panel de expertos e ingenieros invitados.

Criterios de Evaluación

Componente	Peso
Laboratorios Semanales	30%
Examen Parcial de Arquitectura y RAG (Semana 8)	20%
Avance de Investigación y Bitácora de Código	15%

Componente	Peso
Proyecto Final Integrador (Código, Paper y Demo)	35%

User prompt: Proporciona una lista de lecturas académicas, papers clave y documentación técnica para acompañar este syllabus.

Response: **Lecturas Fundamentales y Artículos Académicos (Papers Key)**

Módulo 1: Arquitectura Multimodal y Contexto Extremo

- **Gemini Team, Google.** (2023). *Gemini: A Family of Highly Capable Multimodal Models*. arXiv:2312.11805.
  - **Por qué es clave:** Paper técnico original donde se introduce la arquitectura de entrenamiento nativo multimodal de la familia Gemini (Ultra, Pro, Nano).
- **Gemini Team, Google DeepMind.** (2024). *Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context*. arXiv:2403.05530.
  - **Por qué es clave:** Detalla la arquitectura Mixture-of-Experts (MoE) y el funcionamiento de la ventana de contexto masivo de más de 1M de tokens.
- **Vaswani, A., et al.** (2017). *Attention Is All You Need*. NeurIPS.
  - **Por qué es clave:** Lectura obligatoria para entender los mecanismos de atención sobre los que se construyen los Transformer LMMs.

Módulo 2: Razonamiento Avanzado y Prompting

- **Wei, J., et al.** (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS.
  - **Por qué es clave:** El trabajo pivotal sobre cómo descomponer problemas complejos paso a paso en modelos de gran tamaño.
- **Yao, S., et al.** (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS.
  - **Por qué es clave:** Introduce estrategias de exploración de árboles de decisión y búsquedas tipo BFS/DFS directamente mediante prompts.
- **Mavromatis, C., et al.** (2024). *In-Context Learning for Multimodal Multitask Reasoning*.
  - **Por qué es clave:** Revisa técnicas de *Few-Shot In-Context Learning* aplicadas a entradas mixtas (texto e imagen).

Módulo 3: RAG, Calling de Funciones y Agentes

- **Lewis, P., et al.** (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
  - **Por qué es clave:** El paper pionero que estableció la arquitectura RAG original.
- **Yao, S., et al.** (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR.
  - **Por qué es clave:** Muestra cómo intercalar trazas de razonamiento con la ejecución de acciones/herramientas externas (base para el Function Calling).
- **Schick, T., et al.** (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS.
  - **Por qué es clave:** Analiza cómo los LLMs aprenden a determinar cuándo y cómo invocar APIs externas de forma autónoma.

Módulo 4: Fine-Tuning y MLOps

- **Hu, E. J., et al.** (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR.
  - **Por qué es clave:** La técnica estándar en la industria para el ajuste fino de parámetros de manera eficiente (PEFT).
- **Rafailov, R., et al.** (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model* (DPO). NeurIPS.
  - **Por qué es clave:** Alternativa moderna y estable al RLHF para alinear modelos según preferencias humanas.

Módulo 5: Ética, Seguridad y Gobernanza

- **Google DeepMind.** (2023–2025). *SynthID: Watermarking and detecting AI-generated content*. arXiv / Nature.
  - **Por qué es clave:** Explicación técnica de la inserción de marcas de agua imperceptibles en medios sintéticos (texto e imagen) para garantizar la trazabilidad.
- **Bender, E. M., Gebru, T., et al.** (2021). *On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?*. FAccT.

- **Por qué es clave:** Un análisis crítico fundamental sobre los sesgos, el costo ambiental y la falta de comprensión real de los modelos de lenguaje.

Documentación Técnica y Recursos Prácticos

Recurso	Tipo	Utilidad en el Syllabus
Google GenAI SDK Docs	Documentación Oficial	Referencia para el desarrollo en Python/TypeScript con `google-genai`.
Google AI Studio Documentation	Guía Técnica	Documentación para la construcción y prueba de JSON Schemas y Function Calling.
Vertex AI Generative AI Documentation	Guía Enterprise	Manuales para despliegue de endpoints, tuning y gestión de IAM/seguridad en Google Cloud.
Cookbook de Google Cloud Platform	Repositorio GitHub	Ejemplos de código en producción para RAG con Vertex AI Vector Search y agentes.

Sources

1. (PDF) Gemini: A Family of Highly Capable Multimodal Models - ResearchGate
2. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context - arXiv
3. Gemini: A Family of Highly Capable Multimodal Models - Googleapis.com
4. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context
5. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models - OpenReview
6. Chain of Thought Prompting Elicits Reasoning in Large Language Models - ResearchGate
7. [2510.09263] SynthID-Image: Image watermarking at internet scale - arXiv

---

User prompt: Genera una guía paso a paso para el laboratorio práctico de la Semana 3 sobre Context Caching con la API de Gemini.

Response: Esta guía práctica permite implementar **Context Caching** utilizando el SDK oficial `google-genai` para procesar y consultar grandes volúmenes de datos optimizando latencia y costos.

Prerrequisitos

- Entorno Python 3.10+ configurado.
- Clave de API de Google AI Studio configurada en las variables de entorno.
- Instalación del SDK oficial actualizado:

```
pip install google-genai
```

Guía de Laboratorio

1. **Configuración del entorno e inicialización**

   Paso 1: Autenticación con el SDK unificado

   Carga la clave API e instancia el cliente unificado utilizando el SDK oficial `google-genai`.

   ```
   import os
   from google import genai
   from google.genai import types

   # Inicialización del cliente oficial
   client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))
   ```

2. **Preparación y carga del documento extenso**

   Paso 2: Superar el umbral mínimo de tokens

   Para habilitar el caché, el contenido debe superar el umbral mínimo del modelo (32,768 tokens para Gemini 1.5 Pro / Flash). Subiremos un archivo extenso o un documento grande mediante la Files API.

   ```
   # Carga de un archivo extenso (por ejemplo, un manual o libro técnico)
   large_file = client.files.upload(
       file="manual_tecnico_completo.pdf"
   )

   print(f"Archivo subido exitosamente: {large_file.uri}")
   ```

3. **Creación del recurso de Caché de Contexto**

Paso 3: Definición del Time-To-Live (TTL)

Crea el objeto de caché asignando un tiempo de vida (*TTL*). Esto almacena los tokens procesados en la infraestructura de Google para evitar su re-procesamiento en llamadas subsecuentes.

```
# Creación del caché con un TTL de 1 hora
cache = client.caches.create(
    model="gemini-1.5-flash",
    config=types.CreateCachedContentConfig(
        contents=[large_file],
        system_instruction="Eres un asistente experto analizando el manual técnico adjunto.",
        ttl="3600s", # 1 hora
    )
)

print(f"Caché creado con ID: {cache.name}")
print(f"Expira en: {cache.expire_time}")
```

4 **Consulta utilizando el contexto almacenado**

Paso 4: Ejecución de consultas optimizadas

Realiza preguntas al modelo haciendo referencia explícita al caché creado mediante el parámetro `cached_content`.

```
# Consulta 1 sobre el documento
response = client.models.generate_content(
    model="gemini-1.5-flash",
    contents="¿Cuáles son los procedimientos de seguridad en caso de falla eléctrica descritos en el capít
    config=types.GenerateContentConfig(
        cached_content=cache.name,
    )
)

print(response.text)

# Inspect de uso de tokens para verificar el ahorro
print(f"\nTokens consumidos del caché: {response.usage_metadata.cached_content_token_count}")
print(f"Tokens de prompt no almacenados: {response.usage_metadata.prompt_token_count}")
```

5 **Gestión y monitoreo del Caché**

Paso 5: Listar, extender TTL o eliminar recursos

Administra los recursos activos para optimizar costos de almacenamiento.

```
# Actualizar el TTL si el análisis se prolonga
updated_cache = client.caches.update(
    name=cache.name,
    config=types.UpdateCachedContentConfig(
        ttl="7200s" # Extender a 2 horas
    )
)

# Listar todos los cachés activos
for c in client.caches.list():
    print(f"Cache activo: {c.name} | Expira: {c.expire_time}")

# Limpieza manual al finalizar la práctica
client.caches.delete(name=cache.name)
print("Recurso de caché liberado.")
```

Entregable del Estudiante

1. **Script de Python:** Código funcional con el flujo completo de creación, consulta y liberación del caché.
2. **Reporte de Métricas:** Captura de pantalla del `usage_metadata` mostrando la diferencia de tokens de prompt consumidos con y sin el parámetro `cached_content`.
3. **Análisis Breve:** Explicación técnica de cuándo es financieramente viable aplicar Context Caching considerando las tarifas de almacenamiento por hora frente al costo de procesamiento de input tokens en Gemini.

User prompt: Dame una lista de ejercicios para estudiantes de posgrado utilizando ia de Gemini

Response: Los siguientes ejercicios para posgrado se enfocan en la implementación técnica, arquitectura e investigación utilizando la API de Gemini mediante el SDK unificado `google-genai`.

1. Análisis Multimodal de Datos de Sensores e Imágenes Médicas / Técnicas

- **Objetivo:** Procesar y correlacionar streams de datos heterogéneos en una sola llamada de inferencia.
- **Descripción:** Dado un dataset con imágenes de alta resolución (p. ej., resonancias magnéticas, espectrogramas o mapas topográficos) y un archivo CSV de métricas en formato texto, el estudiante debe enviar ambos insumos a Gemini Pro/Flash usando el SDK.
- **Entregable:** Un script en Python que envíe `[image_part, csv_text]` para extraer inconsistencias entre la imagen y los datos numéricos, devolviendo la respuesta formateada bajo un esquema estricto en JSON (`response_schema`).

2. Implementación de un Agente Autónomo mediante Function Calling ReAct

- **Objetivo:** Construir un bucle de razonamiento y ejecución de herramientas externas (*Tool Use*).
- **Descripción:** Diseñar un agente en Python que responda consultas complejas sobre una base de datos local (p. ej., PostgreSQL o SQLite). El modelo debe declarar funciones (`types.FunctionDeclaration`), interceptar llamadas a funciones (`function_call`), ejecutar las consultas SQL en la base de datos local y devolver la respuesta final sintetizada.
- **Entregable:** Código fuente del agente con manejo de errores cuando la función falla o devuelve datos nulos.

3. Optimización de Costos y Latencia mediante Context Caching

- **Objetivo:** Reducir latencia y consumo de tokens al consultar corpus masivos de literatura científica.
- **Descripción:** El alumno debe subir un libro de texto o un conjunto de 20 papers en PDF (más de 100,000 tokens) a la API. Debe crear un objeto de caché con un `ttl` definido y luego realizar 5 consultas analíticas secuenciales aprovechando `cached_content`.
- **Entregable:** Un reporte comparativo de uso de tokens (`usage_metadata.cached_content_token_count` vs. `prompt_token_count`) y latencia con y sin almacenamiento en caché.

4. Validación y Ejecución de Código con Sandbox Nativo (`Code Execution`)

- **Objetivo:** Utilizar el entorno de ejecución de Python integrado de Gemini para resolver problemas matemáticos, estadísticos o de simulación.
- **Descripción:** Formular problemas cuantitativos (p. ej., resolución de ecuaciones diferenciales, cálculo de invariantes topológicos o simulación Monte Carlo) habilitando la herramienta `code_execution` en la configuración de la llamada.
- **Entregable:** Un notebook que muestre el código Python autogenerado por el modelo, la salida del intérprete en el sandbox y el análisis crítico del estudiante sobre si el código generado es óptimo.

5. Evaluación de Sesgos y Robustez en Búsqueda Grounded (RAG con Google Search)

- **Objetivo:** Evaluar la precisión factual e integración de fuentes de conocimiento externas en tiempo real.
- **Descripción:** Configurar una llamada con la herramienta `google_search` para responder preguntas académicas sobre descubrimientos o publicaciones recientes. El estudiante debe contrastar el texto generado con los metadatos de las fuentes devueltas (*Grounding Metadata*).
- **Entregable:** Un script que parsee las fuentes citadas por el modelo y un análisis de métricas sobre la tasa de precisión (*factuality score*) y atribución correcta de citas.

Sources

1. Google Gen AI Python SDK (googleapis/python-genai) - Context7
2. Function calling with the Gemini API - Interactions API | Google AI for Developers
3. Code execution - Interactions API - Google AI for Developers
4. Gemini API: Function calling with Python - Google Colab
5. Code execution - generateContent API | Google AI for Developers

---